Apache Kafka

De datos almacenados a eventos que se generan continuamente.

BATCH

Archivo completo → procesamiento posterior
Ejemplo: consolidar ventas al final del día.

Datos disponibles antes del análisis

EVENT STREAMING

Cada venta genera un evento que puede publicarse y consumirse sin esperar al cierre.

Eventos disponibles durante la operación

¿Por qué aparece Kafka?

Cuando múltiples sistemas necesitan los mismos eventos con ritmos y objetivos diferentes.

Múltiples destinos

Una venta puede alimentar inventario, monitoreo, analítica y almacenamiento.

Desacoplamiento

El productor publica sin tener que coordinar directamente cada consumidor.

Persistencia y relectura

Los eventos se mantienen según la configuración de retención y pueden releerse.

Kafka transporta y conserva eventos; no reemplaza automáticamente las bases de datos ni los motores analíticos.

¿Dónde utilizar Kafka?

Tres escenarios del material teórico.

COMERCIO

Ventas en sucursales y web → alertas, inventario y análisis comercial.

IoT INDUSTRIAL

Sensores → telemetría, supervisión y detección de anomalías.

MONITOREO

Registros y eventos de aplicaciones → diagnósticos y seguimiento oportuno.

Criterio de elección: flujos continuos + múltiples procesos

Arquitectura básica de Kafka

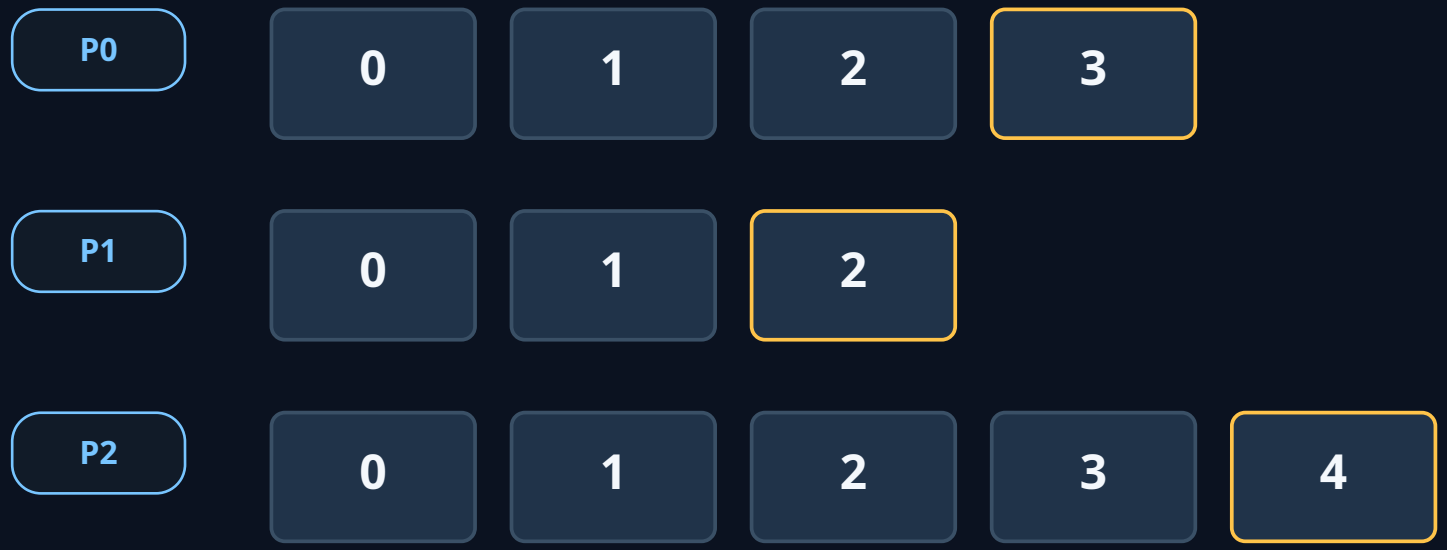
Cada componente tiene una responsabilidad explícita.



El productor no necesita conocer a todos los consumidores.

Particiones y offsets

Un tópico puede dividirse para distribuir almacenamiento y trabajo.



El offset identifica la posición de un evento DENTRO de su partición.

Réplicas y grupos de consumidores

Disponibilidad y paralelismo son mecanismos distintos.

REPLICACIÓN

Mantiene copias de particiones en brokers distintos. Su utilidad depende de la configuración del clúster.

Réplicas ≠ particiones

CONSUMER GROUP

Distribuye particiones entre consumidores de un mismo grupo para procesar en paralelo.

Un grupo: una asignación por partición

Batch, Near Real-Time y tiempo real

Kafka no convierte automáticamente una solución en tiempo real estricto.

BATCH

Datos acumulados. Procesamiento según disponibilidad o programación.

NEAR REAL-TIME

Procesamiento oportuno con cierta latencia aceptable.

REAL-TIME

Restricciones temporales explícitas; exige garantías adicionales.

La latencia depende de toda la arquitectura, no solo del broker.

Un mensaje necesita significado

Las claves, los valores y las marcas temporales permiten interpretar eventos.

```
{
  "id_venta": "V-1048",
  "id_cliente": "C-025",
  "monto": 45990,
  "moneda": "CLP",
  "estado": "confirmada"
}
```

CLAVE

Ej.: id_cliente

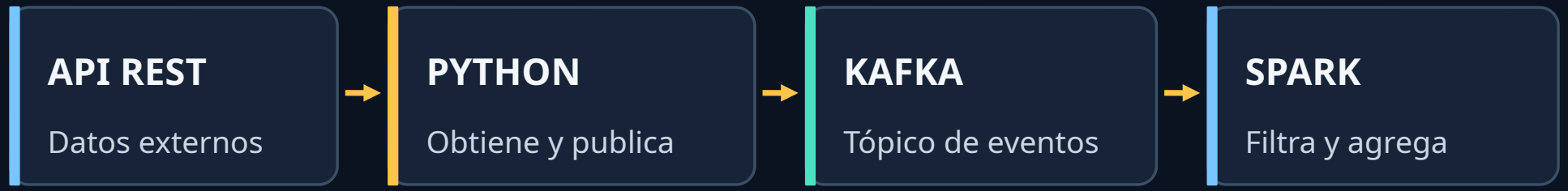
VALOR JSON

Campos estructurados para consumidores.

Timestamp: cuándo ocurrió el evento

De una API a Spark Structured Streaming

Separación de responsabilidades: extracción, distribución y análisis.



APIs

Una consulta devuelve datos; un productor decide qué publicar.

CHECKPOINT

Spark conserva progreso de procesamiento; no elimina duplicados por sí solo.

Kafka en el ecosistema Big Data

Una arquitectura útil cuando la necesidad es mover eventos entre sistemas.

KAFKA

Recibe, almacena temporalmente y distribuye eventos.

SPARK

Transforma, filtra y analiza eventos o datos almacenados.

HDFS / HIVE

Almacenan, organizan y habilitan consultas sobre datos.

Pregunta final: ¿el problema exige una plataforma de eventos o basta una integración más simple?